

Scheduling

Main Points

- Scheduling policy: what to do next, when there are multiple threads ready to run
 - Or multiple packets to send, or web requests to serve, or ...
- Definitions
 - response time, throughput, predictability
- Uniprocessor policies
 - FIFO, SJF, round robin, optimal
 - multilevel feedback as approximation of optimal
- Multiprocessor policies
 - Affinity scheduling, Gang scheduling

Definitions

- Task/Job
 - User request: e.g., mouse click, web request, shell command, ...
- Latency/response time
 - How long does a task take to complete?
- Throughput
 - How many tasks can be done per unit of time?
- Overhead
 - How much extra work is done by the scheduler?
- Fairness
 - How equal is the performance received by different users?
- Predictability
 - How consistent is the performance over time?

More Definitions

- Workload
 - Set of tasks for system to perform
- Preemptive scheduler
 - If we can take resources away from a running task
- Work-conserving
 - Resource is used whenever there is a task to run
- Scheduling algorithm
 - takes a workload as input
 - decides which tasks to do first
 - Performance metric (throughput, latency) as output
 - Only work-conserving schedulers to be considered

First In First Out (FIFO)

- Schedule tasks in the order they arrive
 - Continue running them until they complete or give up the processor
- On what workloads is FIFO particularly bad?

Shortest Job First (SJF)

Two forms:

- non pre-emptive (“senza pre-rilascio”):
 - take the task with the shortest remaining work to do and run it up to its end or until it releases the processor
- pre-emptive (“con pre-rilascio”):
 - run the task with the shortest remaining amount of work to do
 - If a new shorter task wakes up: context switch and run the new task
 - Often called Shortest Remaining Time First (SRTF)

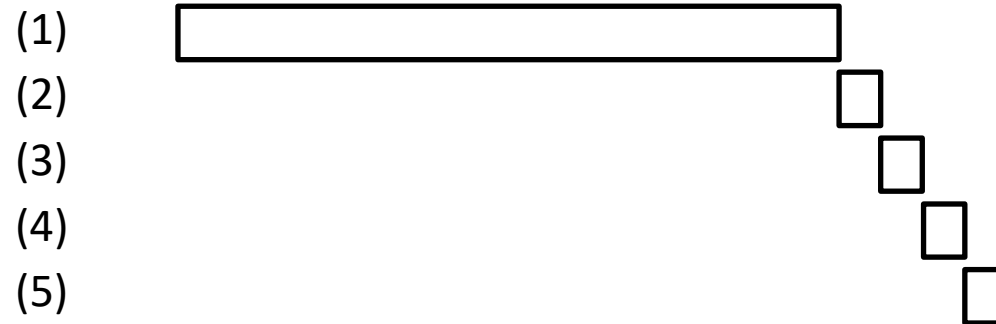
Shortest Job First (SJF)

- Suppose we have five tasks that arrive one right after each other, but the first one is much longer than the others
 - Which completes first in FIFO? Next?
 - Which completes first in SJF? Next?

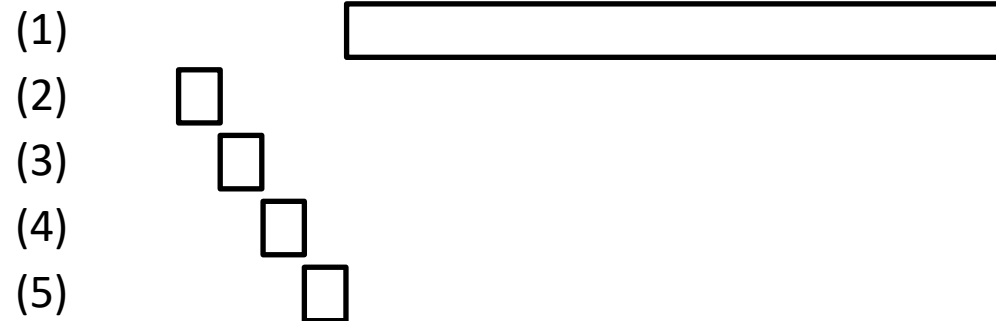
FIFO vs. SJF

Tasks

FIFO



SJF



Time

Shortest Job First

- Claim: SJF is optimal for average response time
 - Why?
- For what workloads is FIFO optimal?
- Pessimal?
- Does SJF have any downsides?

Shortest Job First

SJF minimizes the average response time (turnaround):

4 jobs A,B,C,D with execution time: a, b, c, d

Scheduling sequence: $a \rightarrow b \rightarrow c \rightarrow d$

- turnaround(A) -- a
- turnaround(B) -- $a + b$
- turnaround(C) -- $a + b + c$
- turnaround(D) -- $a + b + c + d$

Average turnaround: $(4a + 3b + 2c + d) / 4$

Minimized iff a, b, c, d are sorted in increasing order

iff = “if and only if”

Shortest Job First

Downsides:

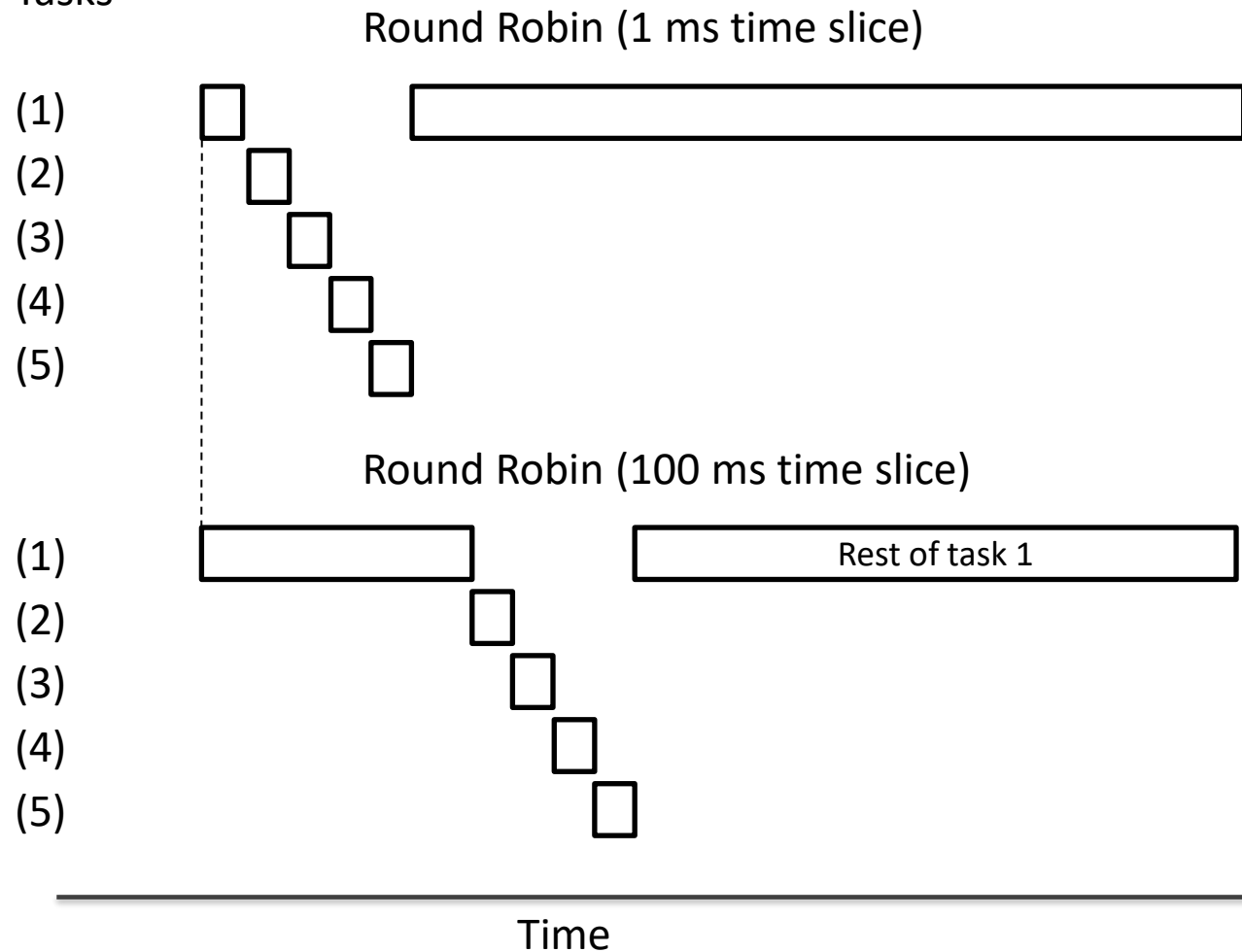
- Starvation ... some task might take forever?
- Variance in response time

Round Robin

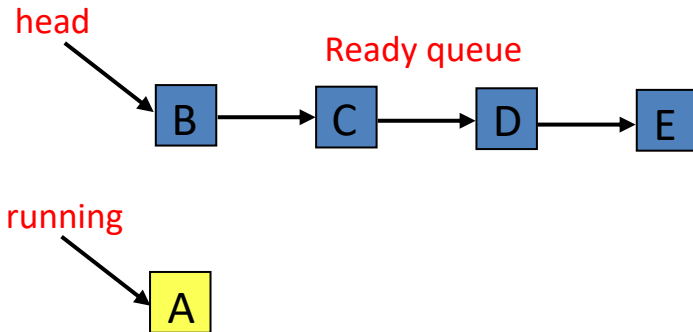
- Each task gets resource for a fixed period of time (time quantum)
 - If task doesn't complete, it goes back in line
- Need to pick a time quantum
 - What if time quantum is too long?
 - Infinite? Then, it falls into FIFO....
 - What if time quantum is too short?
 - One instruction? Too much overhead....

Round Robin

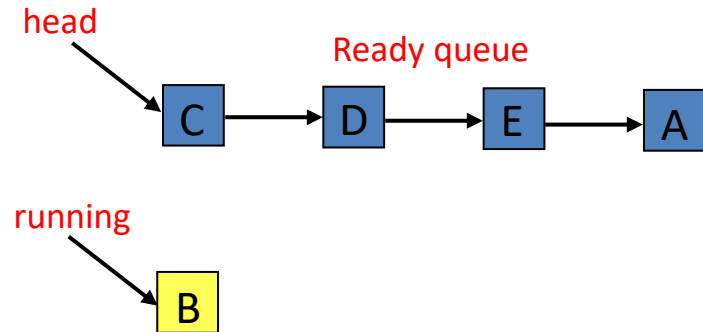
Tasks



Round Robin



(a) A in execution



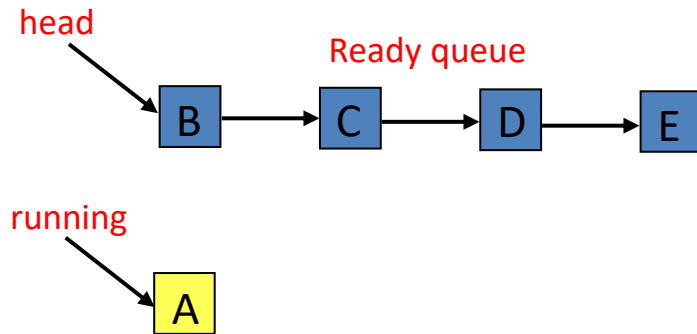
(b) A completes its time share

- Proportionality: turnaround (time spent in the system) proportional to task length
- Response time upper-bounded by the number of processes
 - $\# \text{ ready processes} * \text{time share}$

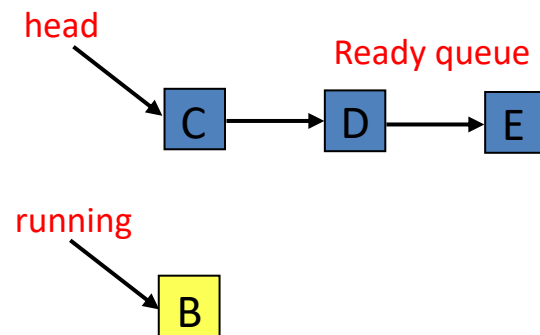
Round Robin

Management of waiting threads

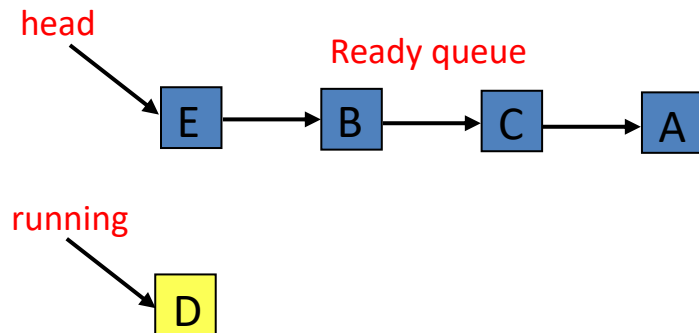
(a) A in execution



(b) A waiting



(c) A resumed



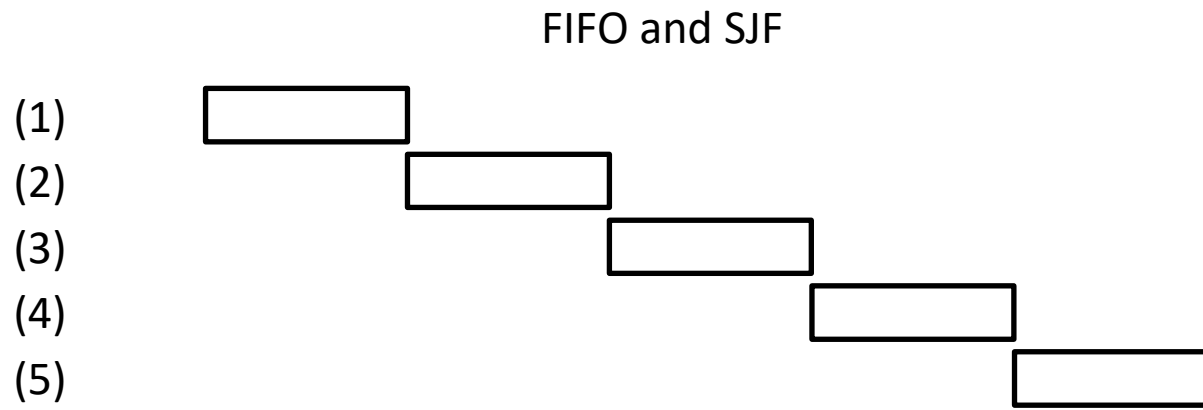
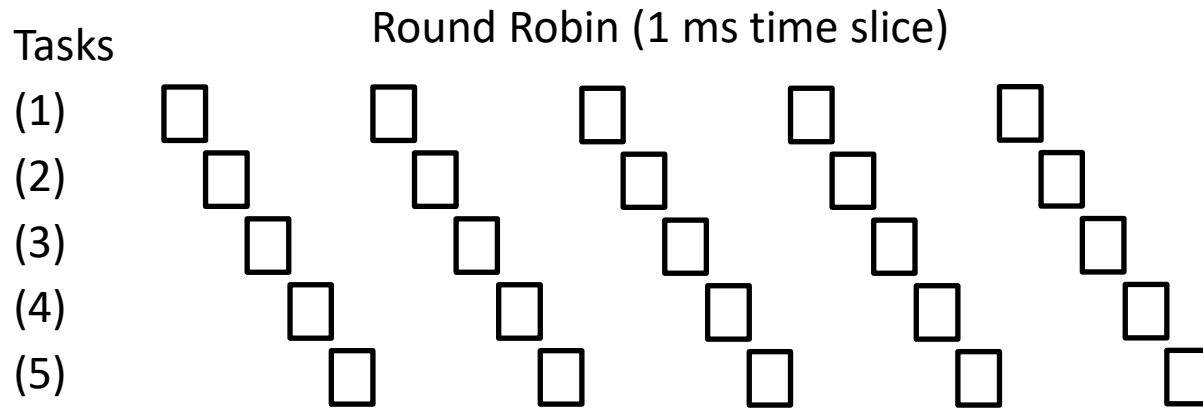
Round Robin

- End of time share signaled by the timer
 - Timer interrupt causes the activation of the scheduler
 - The scheduler restarts the timer
- Scheduler takes over also in case of suspension of the running process
 - Reassigns the CPU and restarts timer
- In current systems time share around 20-120 msec

Round Robin vs. FIFO

- Assuming zero-cost time slice, is Round Robin always better than FIFO?

Round Robin vs FIFO



Time

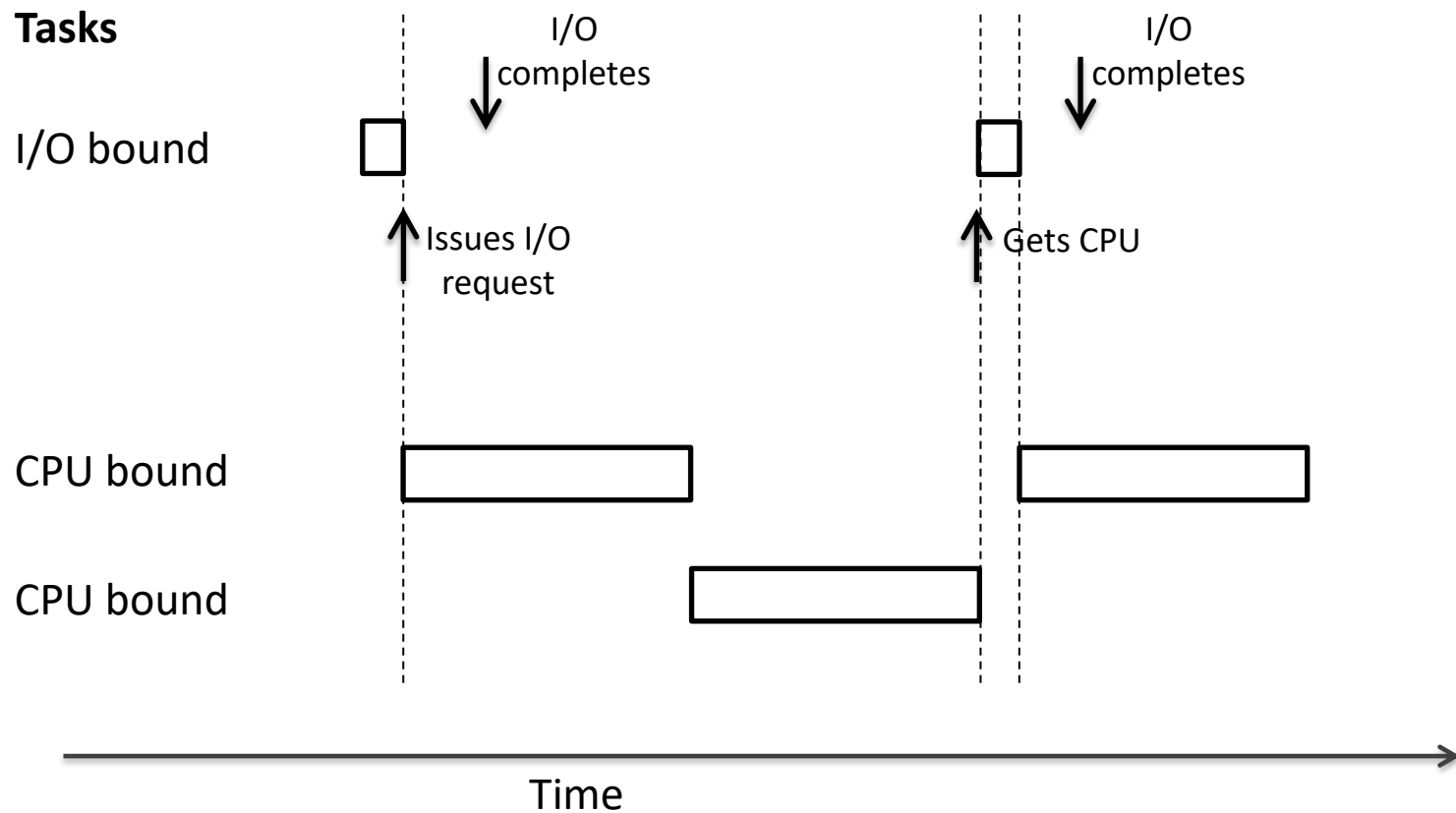
Round Robin = Fairness?

- Is Round Robin always fair?

Mixed Workload

- Let's consider two kind of processes:
 - CPU-bound – long CPU bursts with sparse I/O
 - E.g., number crunching computation
 - I/O bound – short CPU bursts with frequent I/O
 - E.g., highly interactive computation

Mixed Workload



Max-Min Fairness

- How do we balance a mixture of repeating tasks:
 - Some I/O bound, need only a little CPU
 - Some compute bound, can use as much CPU as they are assigned
- Approach: *maximize the minimum allocation given to a task*
 - Schedule the smallest task first
 - Split the remaining time using max-min
 - If all remaining tasks need at least equal share, split evenly

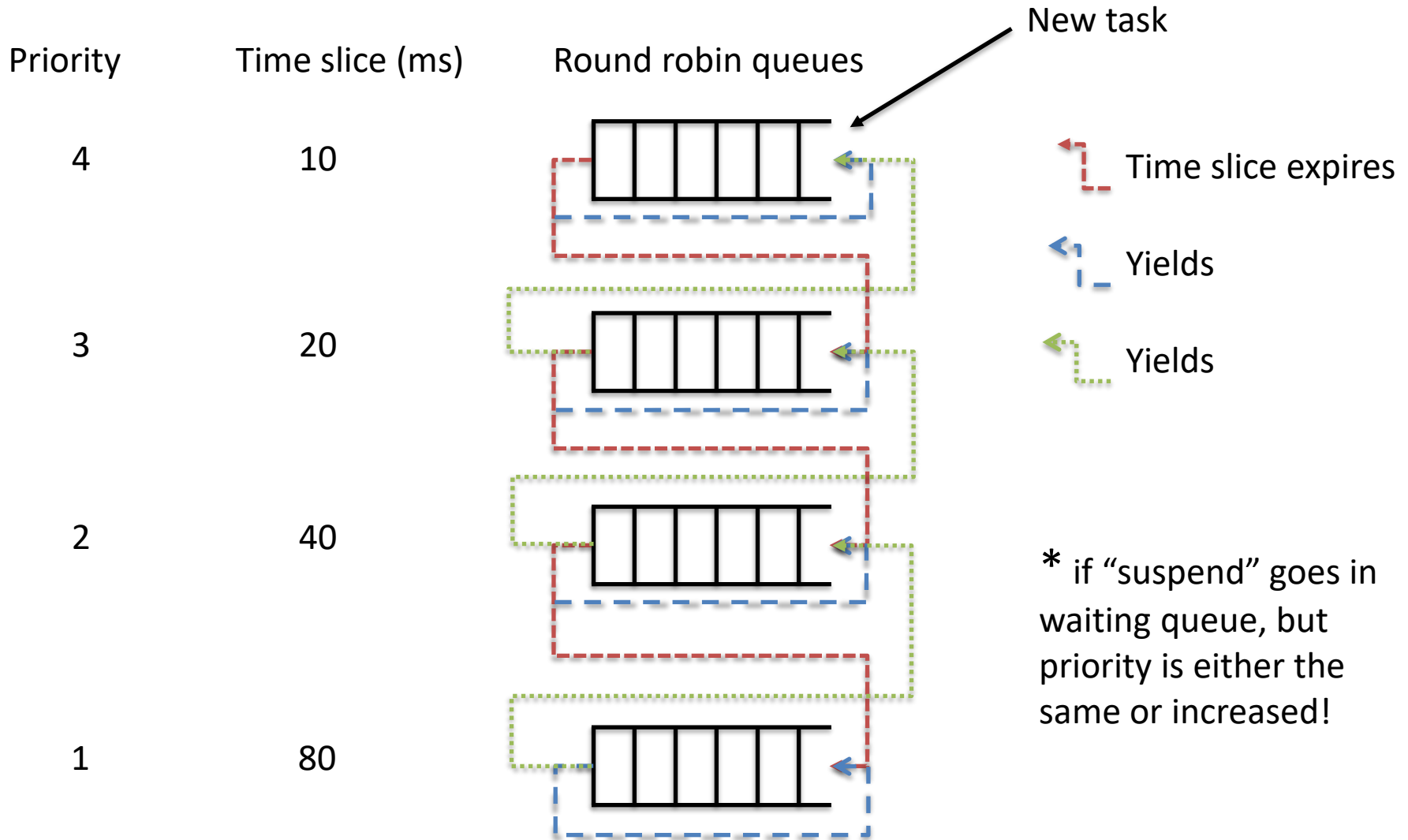
Multi-level Feedback Queue (MFQ)

- Goals:
 - Responsiveness
 - Low overhead
 - Starvation freedom
 - Some tasks are high/low priority
 - Fairness (among equal priority tasks)
- Not perfect at any of them!
 - Used in Linux, Windows, MacOS

MFQ

- Set of Round Robin queues
 - Each queue has a different priority
- High priority queues have short time slices
 - Low priority queues have long time slices
- Scheduler picks first thread in highest priority queue
 - Round robin in each queue

MFQ



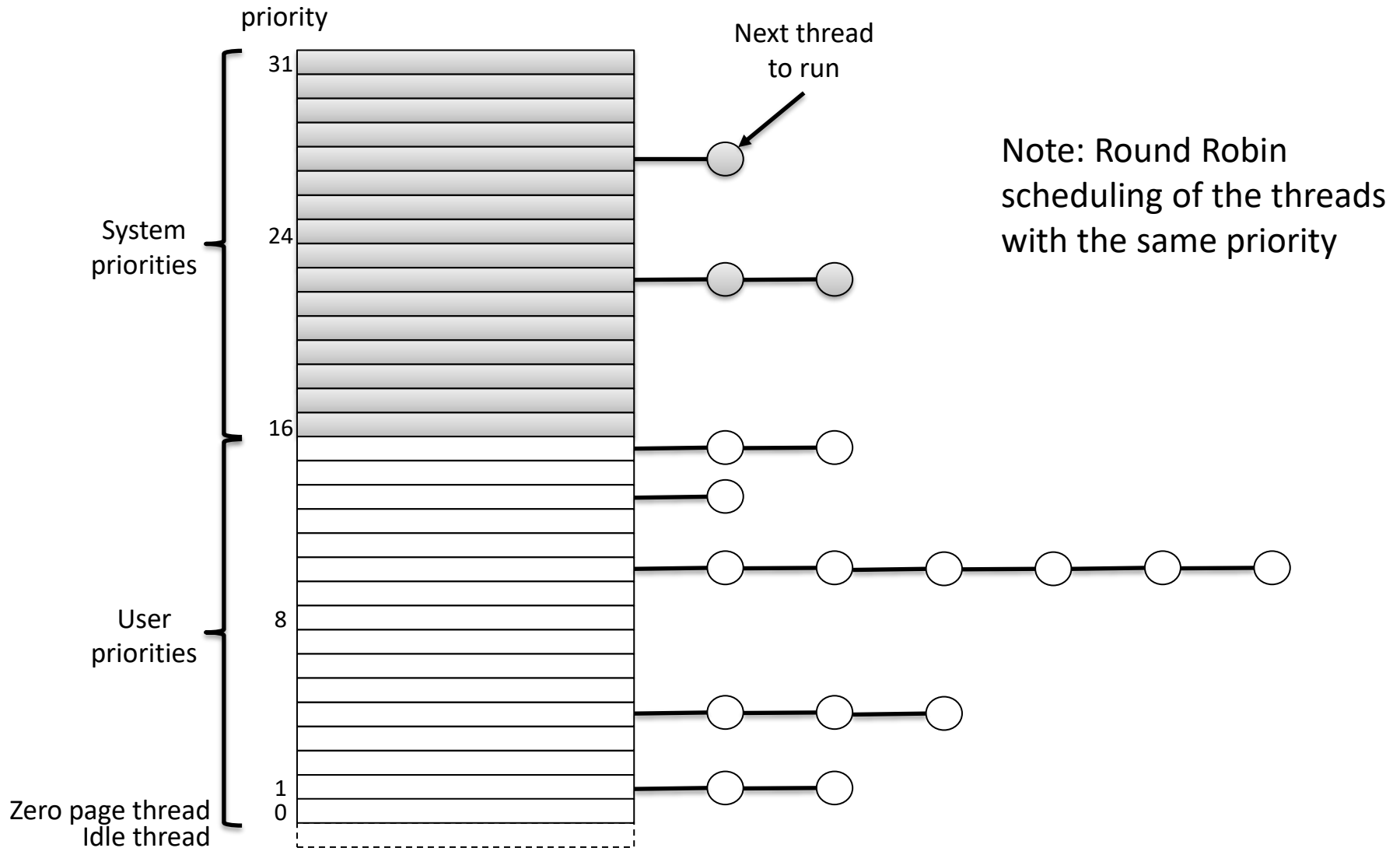
MFQ

- Tasks start in highest priority queue
- If time slice expires, task drops one level
 - i.e. the task has used entirely its time share
 - Task priority is reduced
- If task suspends or yields the processor, it remains in its level (or is bumped up)
 - Task priority is increased
- dynamic priority : tasks may change “habits”:
from CPU-bound to I/O-bound and vice-versa

MFQ

- Starvation with MFQ
 - If the upper queues are always full of I/O-bound processes
- Need for policies to raise up the priority of CPU-bound processes that are starving

Example: scheduling in Windows

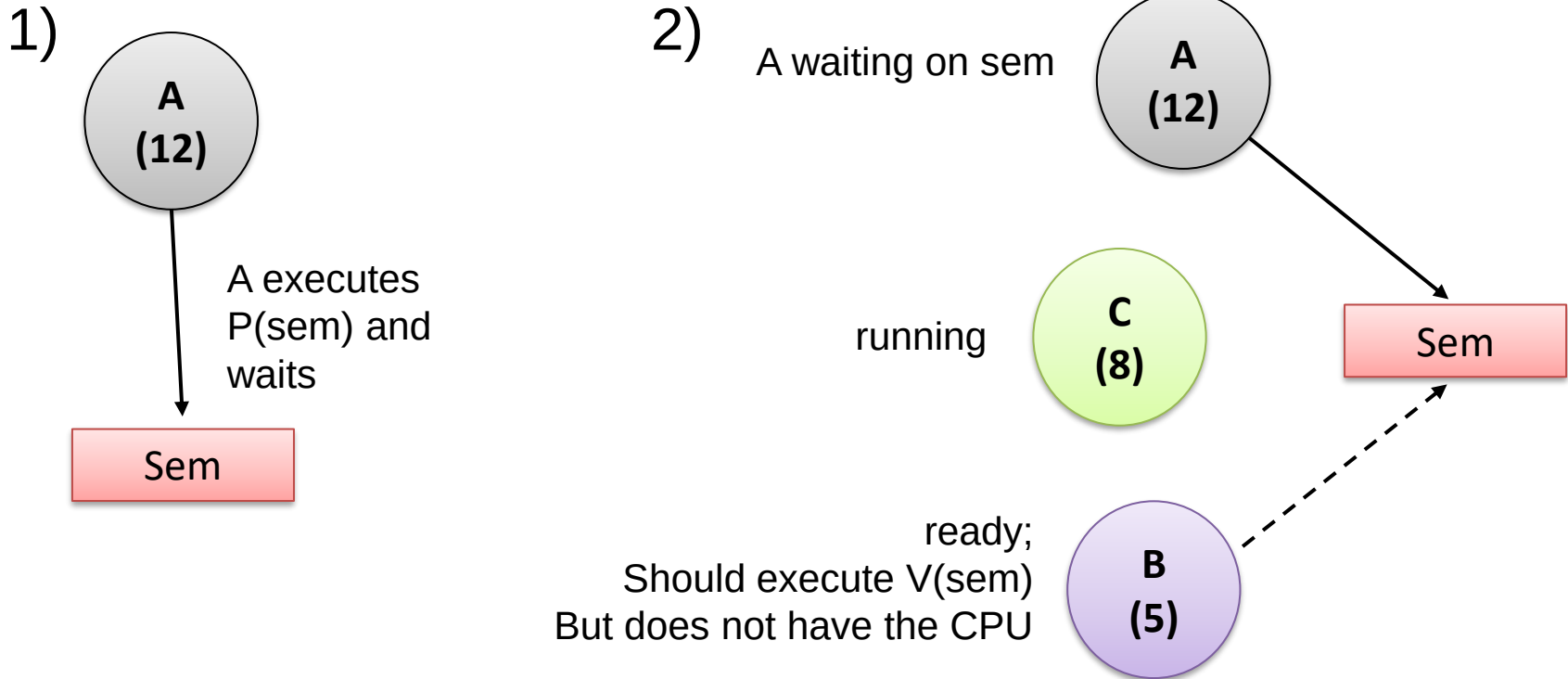


Example: scheduling in Windows

- A new thread starts with priority 8
- Priority raised up if :
 - thread reactivated after I/O operation (disk : +1, Serial line: +6, Keyboard: +8, Audio card: +8, ...)
 - thread reactivated after waiting on a mutex/semaphore (+1 if in background, +2 if in foreground)
 - Thread didn't run for a given amount of time (priority goes to 15 for two time shares)
 - against inversion of priority (see next slide)
- Priority lowered if thread uses all time share (-1)
- When a window goes in foreground the time share of its threads is enlarged

Example: scheduling in Windows

Inversion of priority



Uniprocessor Summary (1)

- FIFO is simple and minimizes overhead.
- If tasks are variable in size, then FIFO can have very poor average response time.
- If tasks are equal in size, FIFO is optimal in terms of average response time.
- Considering only the processor, SJF is optimal in terms of average response time.
- SJF is pessimal in terms of variance in response time.

Uniprocessor Summary (2)

- If tasks are variable in size, Round Robin approximates SJF.
- If tasks are equal in size, Round Robin will have very poor average response time.
- Tasks that intermix processor and I/O do poorly under Round Robin.
- Max-min fairness can improve response time for I/O-bound tasks.
- Round Robin and Max-min fairness both avoid starvation.
- By manipulating the assignment of tasks to priority queues, an MFAQ scheduler can achieve a balance between responsiveness, low overhead, and fairness.

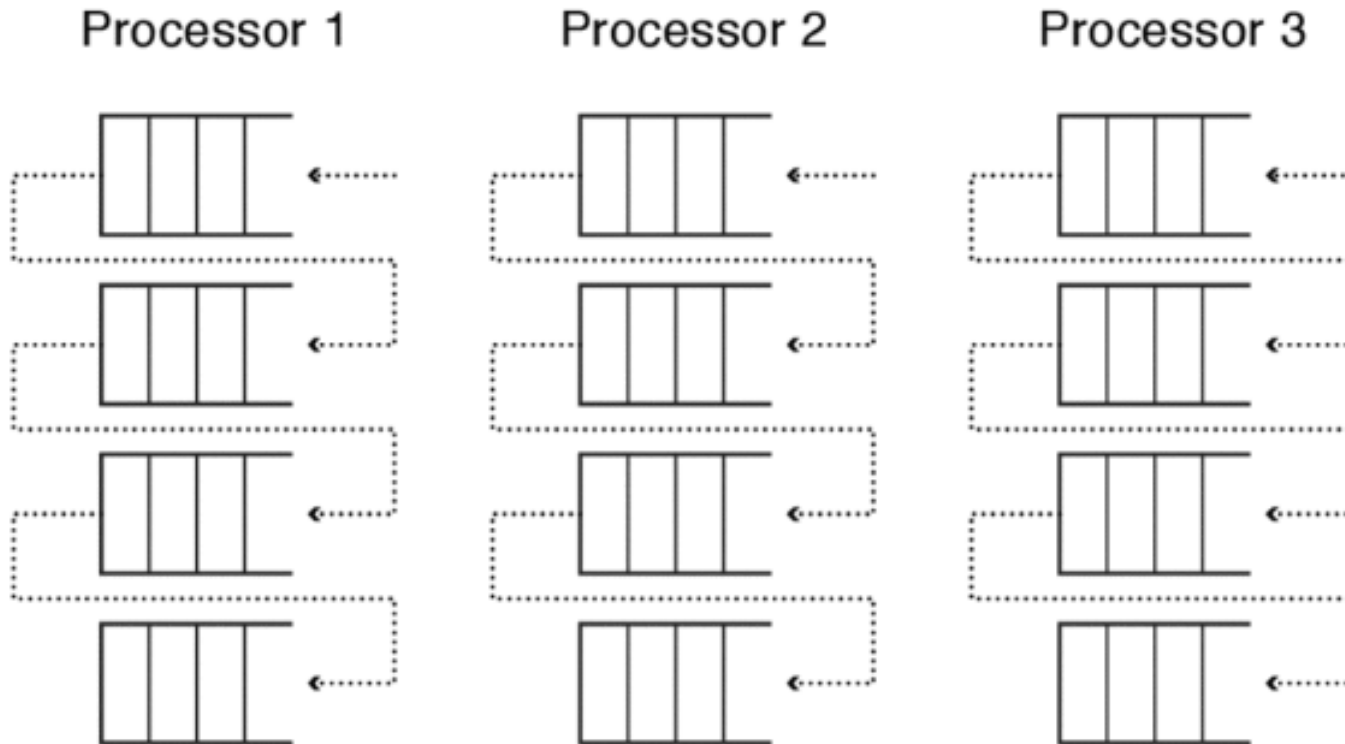
Multiprocessor

- Today, most general-purpose computers are multiprocessors
 - In 2014 we entered in the so-called “multi-core era”
- Many high-end servers have multiple CPUs, each with several cores
 - Today, a single server has $O(100)$ cores
 - Example: IBM Power10 CPU has 15 cores SMT8 (=120 logical cores for the OS). A server can be equipped with several CPUs.
- **Main point for OS:**
 - How do we make effective use of multiple cores for running both sequential tasks as well as highly parallel applications?

Multiprocessor Scheduling

- What would happen if we used MFQ on a multiprocessor?
 - Contention for scheduler spinlock
 - Cache slowdown due to ready list data structure ping-pong from one CPU to another
 - Limited cache reuse: thread's data from last time it ran is often still in its old cache

Per-Processor Multi-level Feedback with Affinity Scheduling



Per-Processor Affinity Scheduling

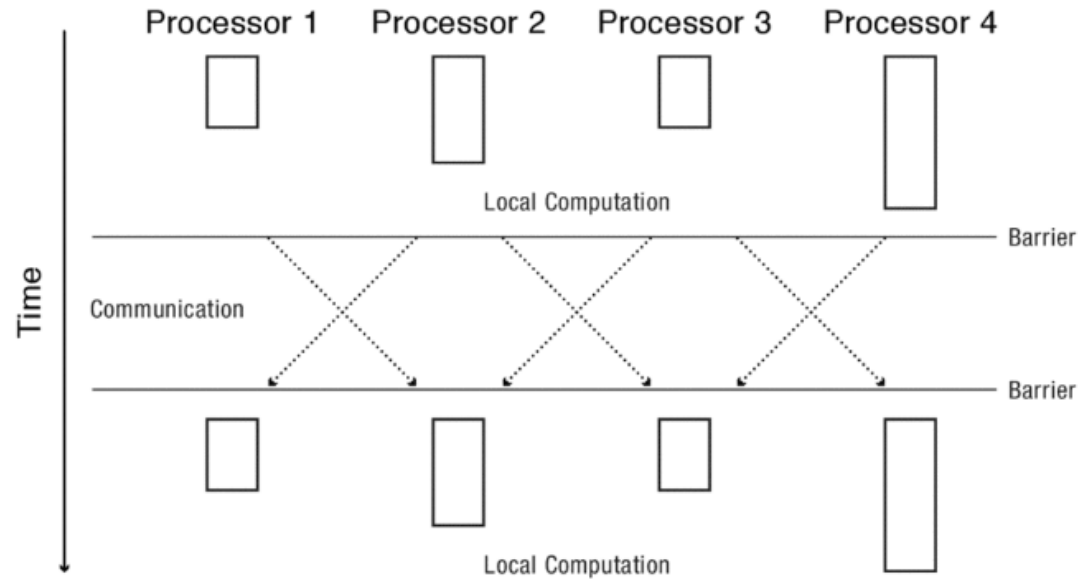
- Each processor has its own (multi-level) ready list
 - Protected by a per-processor spinlock
- Put threads back on the ready list where it had most recently run
 - Ex: when I/O completes, or on CV signal
- Idle processors can steal work from other processors
 - How many jobs is convenient to steal? From whom?
 - When is it convenient to rebalance the workload?

Scheduling Parallel Programs

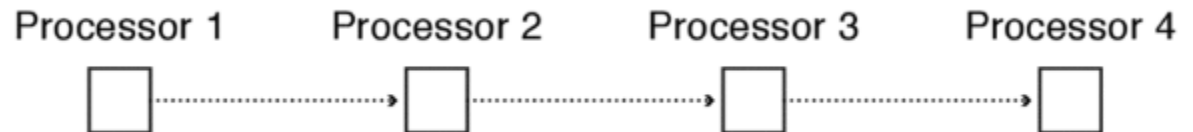
- What happens if one thread gets time-sliced while other threads from the same program are still running?
 - Assuming program uses locks and condition variables, it will still be correct
 - What about performance?

Some Parallel Patterns

Bulk Synchronous Parallel (BSP)



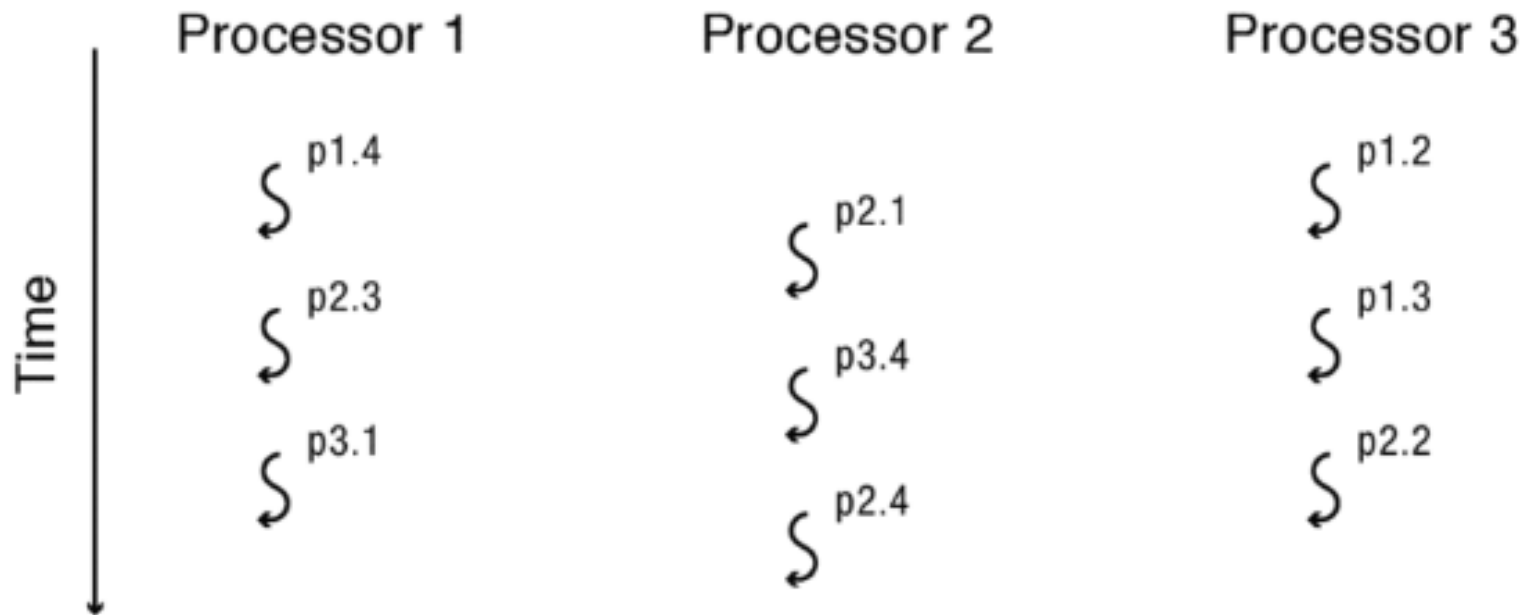
Producer-Consumer pipeline



Preempting one processor can stall all processors!

Scheduling Parallel Programs

Oblivious Scheduling: each processor time-slices its ready list independently of the other processors



$px.y$ = Thread y in process x

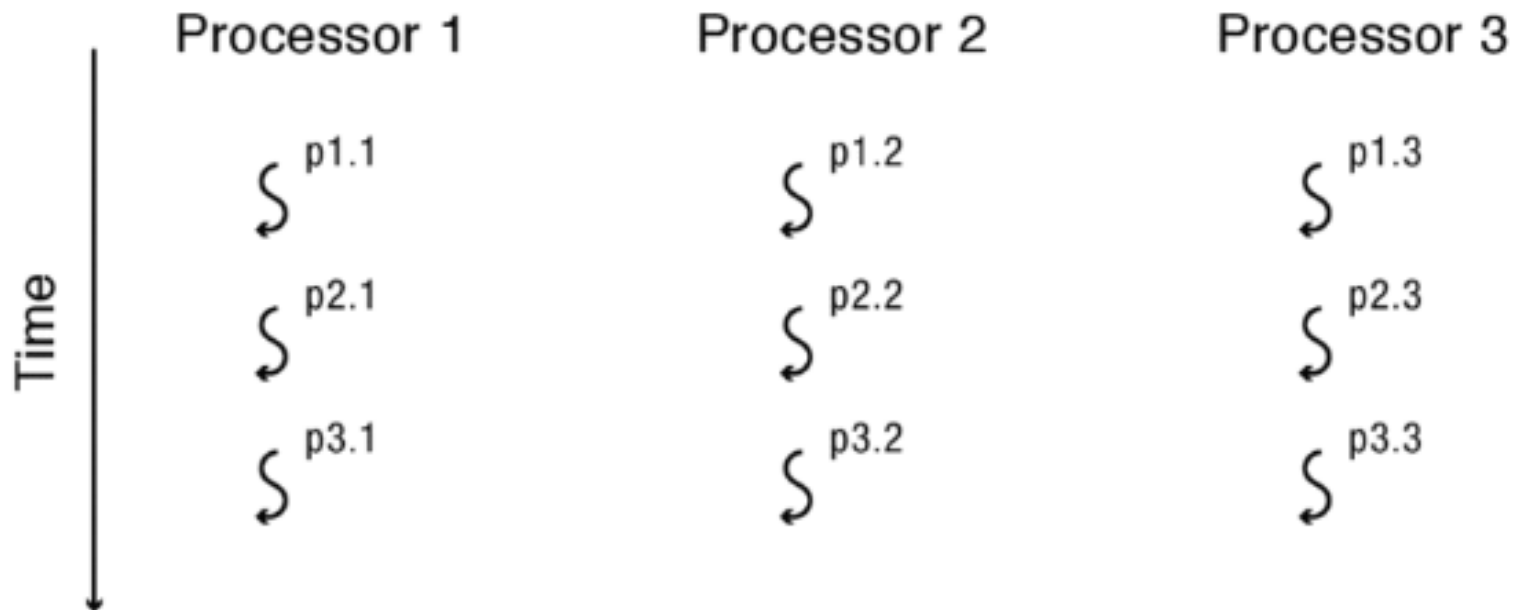
Oblivious Scheduling

Oblivious scheduling potential issues:

- Delay in BSP-like computations
- Delay in Producer-Consumer-like computations
- Critical path delay
- Preemption of lock holder
- I/O delay of some threads

Gang Scheduling

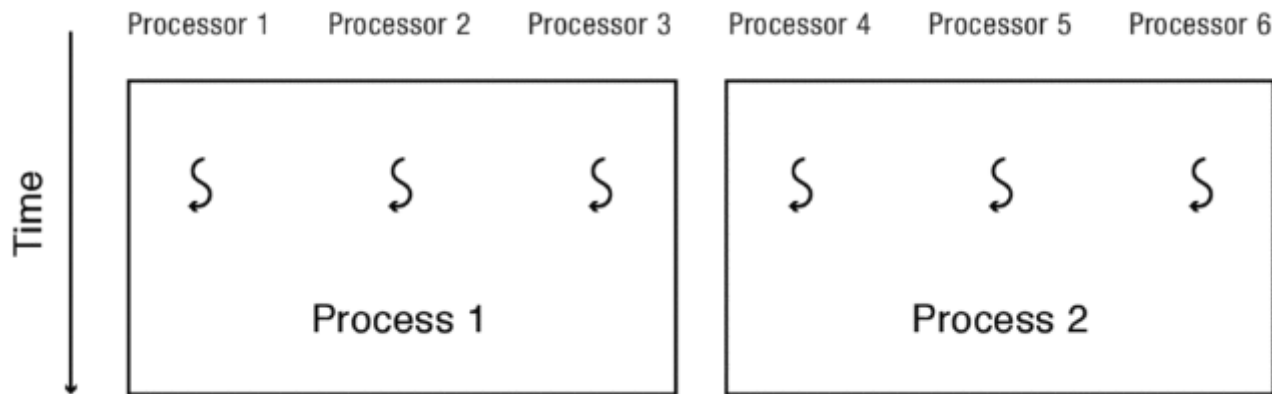
- One solution is to schedule all of the tasks of a program together



px.y = Thread y in process x

Space Sharing

- However, gang scheduling can be inefficient for multiplexing multiple parallel applications



- Allocating different processors to different tasks